

この章では、オープンクラスや、それらがインターフェースとどのように連携するか、そしてKotlinで利用可能なその他の特別な クラスの種類について学びます。

Open classes

インターフェースや抽象クラスを使用できない場合は、クラスを**open**として宣言することで、そのクラスを明示的に継承可能にすることができます。 これを行うには、クラス宣言の前に **open** キーワードを使用します：

```
open class Vehicle(val make: String, val model: String)
```

別のクラスを継承するクラスを作成するには、クラスヘッダーの後にコロンを付け、その後に継承元となる親クラスのコンストラクタを呼び出します。 この例では、**Car** クラスは **Vehicle** クラスを継承しています：

```
open class Vehicle(val make: String, val model: String)

class Car(make: String, model: String, val numberOfDoors: Int) : Vehicle(make,
model)

fun main() {
    // Creates an instance of the Car class
    val car = Car("Toyota", "Corolla", 4)

    // 車の詳細を表示します
    println("Car Info: Make - ${car.make}, Model - ${car.model}, Number of doors -
${car.numberOfDoors}")
    // Car Info: Make - Toyota, Model - Corolla, Number of doors - 4
}
```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-classes-open-class"}
```

通常のクラスインスタンスを作成する場合と同様に、クラスが親クラスを継承している場合は、親クラスのヘッダーで宣言されているすべてのパラメータを初期化する必要があります。したがって、この例では、**Car** クラスの **car** インスタンスが、親クラスのパラメータである **make** と **model** を初期化します。

Overriding inherited behavior

継承された動作のオーバーライド

クラスを継承しつつ、その動作の一部を変更したい場合は、継承された動作をオーバーライドすることができます。

デフォルトでは、親クラスのメンバ関数やプロパティをオーバーライドすることはできません。抽象クラスと同様に、特別なキーワードを追加する必要があります。

Member functions

親クラスの関数をオーバーライドできるようにするには、親クラスでのその関数の宣言の前に **open** キーワードを使用します：

```
open fun displayInfo() {}
```

```
{validate="false"}
```

継承されたメンバ関数をオーバーライドするには、子クラスで関数宣言の前に **override** キーワードを使用します：

```
override fun displayInfo() {}
```

```
{validate="false"}
```

例えば：

```
open class Vehicle(val make: String, val model: String) {
    open fun displayInfo() {
        println("Vehicle Info: Make - $make, Model - $model")
    }
}

class Car(make: String, model: String, val numberOfDoors: Int) : Vehicle(make,
model) {
    override fun displayInfo() {
        println("Car Info: Make - $make, Model - $model, Number of Doors -
$numberOfDoors")
    }
}

fun main() {
    val car1 = Car("Toyota", "Corolla", 4)
    val car2 = Car("Honda", "Civic", 2)

    // オーバーライドされた displayInfo() 関数を使用します
    car1.displayInfo()
    // Car Info: Make - Toyota, Model - Corolla, Number of Doors - 4
    car2.displayInfo()
    // Car Info: Make - Honda, Model - Civic, Number of Doors - 2
}
```

```
{kotlin-runnable="true" id="kotlin-tour-class-override-function"}
```

この例：

- **Vehicle** クラスを継承する **Car** クラスのインスタンスを 2 つ作成します：**car1** と **car2**。
- **Car** クラスの **displayInfo()** 関数をオーバーライドし、ドアの数も出力するようにします。

- `car1` および `car2` のインスタンスに対して、オーバーライドされた `displayInfo()` 関数を呼び出します。

Properties

プロパティ

Kotlinでは、`open`キーワードを使用してプロパティを継承可能にし、後でそれをオーバーライドするというやり方は一般的ではありません。大抵の場合、デフォルトでプロパティが継承可能な抽象クラスやインターフェースを使用します。

オープンクラス内のプロパティは、その子クラスからアクセス可能です。一般的には、新しいプロパティでオーバーライドするよりも、直接アクセスするほうが望ましいです。

たとえば、後で上書きしたい `transmissionType` というプロパティがあるとします。プロパティのオーバーライドの構文は、メンバ関数のオーバーライドの構文とまったく同じです。次のようにすることができます：

```
open class Vehicle(val make: String, val model: String) {
    open val transmissionType: String = "Manual"
}

class Car(make: String, model: String, val numberOfDoors: Int) : Vehicle(make,
model) {
    override val transmissionType: String = "Automatic"
}
```

しかし、これは望ましい方法とは言えません。その代わりに、継承可能なクラスのコンストラクタにそのプロパティを追加し、子クラス `Car` を作成する際にその値を宣言することができます：

```
open class Vehicle(val make: String, val model: String, val transmissionType:
String = "Manual")

class Car(make: String, model: String, val numberOfDoors: Int) : Vehicle(make,
model, "Automatic")
```

プロパティをオーバーライドするのではなく、直接アクセスすることで、コードがよりシンプルで読みやすくなります。親クラスでプロパティを一度宣言し、その値をコンストラクタを通じて渡すことで、子クラスでの不必要なオーバーライドを回避できます。

クラスの継承やクラスの動作のオーバーライドに関する詳細については、[継承](#)を参照してください。

Open classes and interfaces

Openクラスとインターフェース

あるクラスを継承し、**かつ**複数のインターフェースを実装するクラスを作成することができます。この場合、インターフェースを列挙する前に、コロン (:) の後に親クラスを先に宣言する必要があります：

```
// インターフェースを定義する
interface EcoFriendly {
    val emissionLevel: String
}

interface ElectricVehicle {
    val batteryCapacity: Double
}

// 親クラス（基底クラス）
open class Vehicle(val make: String, val model: String)

// 子クラス（派生クラス）
open class Car(make: String, model: String, val numberOfDoors: Int) :
    Vehicle(make, model)

// Car を継承し、2つのインターフェースを実装する新しいクラス
class ElectricCar(
    make: String,
    model: String,
    numberOfDoors: Int,
    val capacity: Double,
    val emission: String
) : Car(make, model, numberOfDoors), EcoFriendly, ElectricVehicle {
    override val batteryCapacity: Double = capacity
    override val emissionLevel: String = emission
}
```

Special classes

特別なクラス

Kotlinには、抽象クラス、オープンクラス、データクラスに加え、特定の挙動を制限したり、小さなオブジェクトの作成によるパフォーマンスへの影響を軽減したりするなど、さまざまな目的のために設計された特殊な種類のクラスが用意されています。

Sealed classes

シールドクラス

継承を制限したい場合もあるでしょう。これは、シールドクラスを使って実現できます。シールドクラスは、[抽象クラス](#)の特殊なタイプです。クラスを「sealed」と宣言すると、そのクラスから派生する子クラスは、同じパッケージ内でのみ作成できるようになります。このスコープの外では、sealed クラスから継承することはできません。

パッケージとは、関連するクラスや関数を含むコードの集合体であり、通常は1つのディレクトリ内に収められています。Kotlin のパッケージについて詳しく知りたい場合は、[パッケージとインポート](#)を参照してください。

{style="tip"}

シールドクラスを作成するには、`sealed` キーワードを使用します：

```
sealed class Mammal
```

シールドクラスは、`when` 式と組み合わせると特に有用です。 `when` 式を使用することで、考えられるすべての子クラスに対する挙動を定義することができます。例えば：

```
sealed class Mammal(val name: String)

class Cat(val catName: String) : Mammal(catName)

class Human(val humanName: String, val job: String) : Mammal(humanName)

fun greetMammal(mammal: Mammal): String {
    when (mammal) {      // mammalが<is演算子 (型)>でHuman型かCat型か判定
        is Human -> return "Hello ${mammal.name}; You're working as a
${mammal.job}"

        is Cat -> return "Hello ${mammal.name}"
    }
}

fun main() {
    println(greetMammal(Cat("Snowy")))
    // Hello Snowy
}
```

{kotlin-runnable="true" id="kotlin-tour-sealed-classes"}

この例では：

- `Mammal` という名前のシールドクラスがあり、そのコンストラクタには `name` パラメータがあります。
- `Cat` クラスは `Mammal` シールドクラスを継承しており、自身のコンストラクタ内の `catName` パラメータを、`Mammal` クラスの `name` パラメータとして使用しています。
- `Human` クラスは `Mammal` シールドクラスを継承しており、自身のコンストラクタ内の `humanName` パラメータを、`Mammal` クラスの `name` パラメータとして使用しています。また、このクラスのコンストラクタには `job` パラメータも含まれています。
- The `greetMammal()` function accepts an argument of `Mammal` type and returns a string.
- `greetMammal()` 関数の本体には、`is` 演算子を使用して `mammal` の型を確認し、どのアクションを実行するかを決定する `when` 式があります。
- `main()` 関数は、`Cat` クラスのインスタンスと、`Snowy` という名前の `name` パラメータを引数として、`greetMammal()` 関数を呼び出します。

このツアーでは、[Null Safety](#)の章で、`is` 演算子についてさらに詳しく解説しています。

{style="tip"}

シールドクラスとその推奨される使用例に関する詳細については、[シールドクラスとインターフェース](#)を参照してください。

Enum classes

Enum クラス

列挙型クラスは、クラス内で有限個の異なる値の集合を表現したい場合に役立ちます。列挙型クラスには列挙定数が含まれており、これらの列挙定数自体がその列挙型クラスのインスタンスとなります。

列挙型クラスを作成するには、`enum` キーワードを使用します：

```
enum class State
```

例えば、プロセスのさまざまな状態を含む列挙型クラスを作成したいとしましょう。各列挙定数は、コンマ、`,`で区切らなければなりません：

```
enum class State {  
    IDLE, RUNNING, FINISHED  
}
```

`State` 列挙クラスには、`IDLE`、`RUNNING`、`FINISHED` という列挙定数があります。列挙定数にアクセスするには、クラス名の後に `.` を付け、その後に列挙定数の名前を指定します。

```
val state = State.RUNNING
```

この列挙型クラスを `when` 式と組み合わせて使用することで、列挙定数の値に応じて実行するアクションを定義できます：

```
enum class State {  
    IDLE, RUNNING, FINISHED  
}  
  
fun main() {  
    val state = State.RUNNING  
    val message = when (state) {  
        State.IDLE -> "It's idle"  
        State.RUNNING -> "It's running"  
        State.FINISHED -> "It's finished"  
    }  
    println(message)  
    // It's running  
}
```

```
{kotlin-runnable="true" id="kotlin-tour-enum-classes"}
```

列挙型クラスも、通常のクラスと同様に、プロパティやメンバ関数を持つことができます。

たとえば、HTMLを扱っていて、いくつかの色を含む列挙型クラスを作成したい場合を考えてみましょう。各色に、RGB値を16進数で表した`rgb`というプロパティを持たせたいとします。列挙定数を作成する際は、このプロパティを使用して初期化する必要があります：

```
enum class Color(val rgb: Int) {
    RED(0xFF0000),
    GREEN(0x00FF00),
    BLUE(0x0000FF),
    YELLOW(0xFFFF00)
}
```

Kotlinでは16進数は整数として扱われるため、`rgb` プロパティの型は`String`ではなく`Int`になります。

```
{style="note"}
```

このクラスにメンバ関数を追加するには、列挙定数とセミコロン `;` で区切ります：

```
enum class Color(val rgb: Int) {
    RED(0xFF0000),
    GREEN(0x00FF00),
    BLUE(0x0000FF),
    YELLOW(0xFFFF00);

    fun containsRed() = (this.rgb and 0xFF0000 != 0)
}

fun main() {
    val red = Color.RED

    // 列挙定数に対して containsRed() 関数を呼び出す
    println(red.containsRed())
    // true

    // Calls containsRed() function on enum constants via class names
    println(Color.BLUE.containsRed())
    // false

    println(Color.YELLOW.containsRed())
    // true
}
```

```
{kotlin-runnable="true" id="kotlin-tour-interface-enum-classes-members"}
```

この例では、`containsRed()` メンバ関数は、`this` キーワードを使用して列挙定数の `rgb` プロパティの値にアクセスし、その16進数値の先頭ビットに `FF` が含まれているかどうかを確認して、ブール値を返します。

詳細については、[列挙型クラス](#)を参照してください。

Inline value classes

インライン値型クラス

コードを書く際、クラスから小さなオブジェクトを作成し、それをほんの短時間だけ使用したい場合があるかもしれません。このアプローチは、パフォーマンスに影響を与える可能性があります。インライン値(あたい)クラスは、このパフォーマンスへの影響を回避する特殊な種類のクラスです。ただし、これらは値のみを含めることができます。

インライン型値クラスを作成するには、`value` キーワードと `@JvmInline` アノテーションを使用します：

```
@JvmInline
value class Email
```

`@JvmInline` アノテーションは、コンパイル時にコードを最適化するように Kotlin に指示します。詳細については、[アノテーション](#)を参照してください。

{style="tip"}

インライン値型クラスには、クラスのヘッダー内で初期化されたプロパティが1つだけ**必ず**存在しなければなりません。

たとえば、メールアドレスを収集するクラスを作成したいとしましょう：

```
// address プロパティは、クラスのヘッダー内で初期化されます。
@JvmInline
value class Email(val address: String)

fun sendEmail(email: Email) {
    println("Sending email to ${email.address}")
}

fun main() {
    val myEmail = Email("example@example.com")
    sendEmail(myEmail)
    // Sending email to example@example.com
}
```

{kotlin-runnable="true" id="kotlin-tour-inline-value-class"}

この例では：

- `Email` は、クラスヘッダーに `address` というプロパティを 1 つ持つインライン値クラスです。
- `sendEmail()` 関数は、型が `Email` のオブジェクトを受け取り、文字列を標準出力に出力します。
- `main()` 関数では：
 - `Email` クラスのインスタンスを `myEmail` という名前で作成します。

- `myEmail` オブジェクトの `sendEmail()` 関数を呼び出します。

インライン値型クラスを使用することで、そのクラスをインライン化し、オブジェクトを作成することなくコード内で直接使用できるようになります。これにより、メモリ使用量を大幅に削減し、コードの実行時のパフォーマンスを向上させることができます。

インライン値型クラスに関する詳細については、[インライン値型クラス](#)を参照してください。

演習 {completion-point="true"}

課題 1 {initial-collapse-state="collapsed" collapsible="true" id="special-classes-exercise-1"}

あなたは配送サービスを運営しており、荷物の配送状況を追跡する手段を必要としています。

`DeliveryStatus` という名前のシールドクラスを作成し、以下のステータス（`Pending`、`InTransit`、`Delivered`、`Canceled`）を表すデータクラスを含めてください。`main()` 関数内のコードが正常に実行されるように、`DeliveryStatus` クラスの宣言を完成させてください：

|---|---|

```
sealed class // Write your code here

fun printDeliveryStatus(status: DeliveryStatus) {
    when (status) {
        is DeliveryStatus.Pending -> {
            println("The package is pending pickup from ${status.sender}.")
        }
        is DeliveryStatus.InTransit -> {
            println("The package is in transit and expected to arrive by
${status.estimatedDeliveryDate}.")
        }
        is DeliveryStatus.Delivered -> {
            println("The package was delivered to ${status.recipient} on
${status.deliveryDate}.")
        }
        is DeliveryStatus.Canceled -> {
            println("The delivery was canceled due to: ${status.reason}.")
        }
    }
}

fun main() {
    val status1: DeliveryStatus = DeliveryStatus.Pending("Alice")
    val status2: DeliveryStatus = DeliveryStatus.InTransit("2024-11-20")
    val status3: DeliveryStatus = DeliveryStatus.Delivered("2024-11-18", "Bob")
    val status4: DeliveryStatus = DeliveryStatus.Canceled("Address not found")

    printDeliveryStatus(status1)
    // The package is pending pickup from Alice.
    printDeliveryStatus(status2)
    // The package is in transit and expected to arrive by 2024-11-20.
    printDeliveryStatus(status3)
    // The package was delivered to Bob on 2024-11-18.
```

```
    printDeliveryStatus(status4)
    // The delivery was canceled due to: Address not found.
}
```

```
{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-special-classes-exercise-1"}
```

```
|---|---|
```

```
sealed class DeliveryStatus {
    data class Pending(val sender: String) : DeliveryStatus()
    data class InTransit(val estimatedDeliveryDate: String) : DeliveryStatus()
    data class Delivered(val deliveryDate: String, val recipient: String) :
DeliveryStatus()
    data class Canceled(val reason: String) : DeliveryStatus()
}

fun printDeliveryStatus(status: DeliveryStatus) {
    when (status) {
        is DeliveryStatus.Pending -> {
            println("The package is pending pickup from ${status.sender}.")
        }
        is DeliveryStatus.InTransit -> {
            println("The package is in transit and expected to arrive by
${status.estimatedDeliveryDate}.")
        }
        is DeliveryStatus.Delivered -> {
            println("The package was delivered to ${status.recipient} on
${status.deliveryDate}.")
        }
        is DeliveryStatus.Canceled -> {
            println("The delivery was canceled due to: ${status.reason}.")
        }
    }
}

fun main() {
    val status1: DeliveryStatus = DeliveryStatus.Pending("Alice")
    val status2: DeliveryStatus = DeliveryStatus.InTransit("2024-11-20")
    val status3: DeliveryStatus = DeliveryStatus.Delivered("2024-11-18", "Bob")
    val status4: DeliveryStatus = DeliveryStatus.Canceled("Address not found")

    printDeliveryStatus(status1)
    // The package is pending pickup from Alice.
    printDeliveryStatus(status2)
    // The package is in transit and expected to arrive by 2024-11-20.
    printDeliveryStatus(status3)
    // The package was delivered to Bob on 2024-11-18.
    printDeliveryStatus(status4)
    // The delivery was canceled due to: Address not found.
}
```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-special-classes-solution-1"}

課題 2 {initial-collapse-state="collapsed" collapsible="true" id="special-classes-exercise-2"}

プログラムでは、さまざまなステータスや種類のエラーに対応できるようにする必要があります。データクラスやオブジェクトで宣言されているさまざまなステータスを取得するための、シールドクラスが用意され

ています。以下のコードを完成させてください。`Problem` という名前の列挙型クラスを作成し、`NETWORK`、`TIMEOUT`、`UNKNOWN` という異なる問題タイプを表すようにしてください。

|---|---|

```
sealed class Status {
    data object Loading : Status()
    data class Error(val problem: Problem) : Status() {
        // Write your code here
    }

    data class OK(val data: List<String>) : Status()
}

fun handleStatus(status: Status) {
    when (status) {
        is Status.Loading -> println("Loading...")
        is Status.OK -> println("Data received: ${status.data}")
        is Status.Error -> when (status.problem) {
            Status.Error.Problem.NETWORK -> println("Network issue")
            Status.Error.Problem.TIMEOUT -> println("Request timed out")
            Status.Error.Problem.UNKNOWN -> println("Unknown error occurred")
        }
    }
}

fun main() {
    val status1: Status = Status.Error(Status.Error.Problem.NETWORK)
    val status2: Status = Status.OK(listOf("Data1", "Data2"))

    handleStatus(status1)
    // Network issue
    handleStatus(status2)
    // Data received: [Data1, Data2]
}
```

{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-special-classes-exercise-2"}

|---|---|

```
sealed class Status {
    data object Loading : Status()
    data class Error(val problem: Problem) : Status() {
        enum class Problem {
            NETWORK,
            TIMEOUT,
            UNKNOWN
        }
    }
}
```

```
data class OK(val data: List<String>) : Status()
}

fun handleStatus(status: Status) {
    when (status) {
        is Status.Loading -> println("Loading...")
        is Status.OK -> println("Data received: ${status.data}")
        is Status.Error -> when (status.problem) {
            Status.Error.Problem.NETWORK -> println("Network issue")
            Status.Error.Problem.TIMEOUT -> println("Request timed out")
            Status.Error.Problem.UNKNOWN -> println("Unknown error occurred")
        }
    }
}

fun main() {
    val status1: Status = Status.Error(Status.Error.Problem.NETWORK)
    val status2: Status = Status.OK(listOf("Data1", "Data2"))

    handleStatus(status1)
    // Network issue
    handleStatus(status2)
    // Data received: [Data1, Data2]
}
```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-special-classes-solution-2"}

- [Previous step](#)
- [Next step](#)